Chapter 4

-Call-by-value
-Call-by-reference

Learning Objectives

③ Parameters

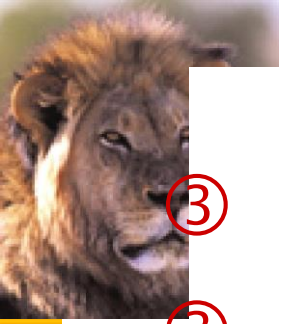
- Call-by-value
- Call-by-reference
- Mixed parameter-lists
- Overloading and Default Arguments
- Examples, Rules
- Testing and Debugging Functions
- assert Macro
- Stubs, Drivers

Parameters

- Two methods of passing arguments as parameters
- Call-by-value
- ‘copy’ of value is passed
- Call-by-reference
- ‘address of’ actual argument is passed

Call-by-Value Parameters

- Copy of actual argument passed



- Considered ‘local variable’ inside function
- If modified, only ‘local copy’ changes
- Function has no access to ‘actual argument’ from caller
- This is the default method
- Used in all examples thus far

Call-by-Value Example

Display 4.1, page 136

Display 4.1 Formal Parameter Used as a Local Variable *(part 1 of 2)*

```
1  //Law office billing program.
2  #include <iostream>
3  using namespace std;

4  const double RATE = 150.00; //Dollars per quarter hour.

5  double fee(int hoursWorked, int minutesWorked);
6  //Returns the charges for hoursWorked hours and
7  //minutesWorked minutes of legal services.

8  int main( )
9  {
10     int hours, minutes;
11     double bill;

12     cout << "Welcome to the law office of\n"
13          << "Dewey, Cheatham, and Howe.\n"
14          << "The law office with a heart.\n"
15          << "Enter the hours and minutes"
16          << " of your consultation:\n";
17     cin >> hours >> minutes;

18     bill = fee(hours, minutes);

19     cout.setf(ios::fixed);
20     cout.setf(ios::showpoint);
21     cout.precision(2);
22     cout << "For " << hours << " hours and " << minutes
23          << " minutes, your bill is $" << bill << endl;

24     return 0;
25 }
```

The value of minutes is not changed by the call to fee.

Call-by-Value Example Cont'd

Display 4.1, page 136

Display 4.1 Formal Parameter Used as a Local Variable *(part 2 of 2)*

```
26  double fee(int hoursWorked, int minutesWorked)
27  {
28      int quarterHours;

29      minutesWorked = hoursWorked*60 + minutesWorked;
30      quarterHours = minutesWorked/15;
31      return (quarterHours*RATE);
32  }
```

minutesWorked is a local variable initialized to the value of minutes.

SAMPLE DIALOGUE

```
Welcome to the law office of
Dewey, Cheatham, and Howe.
The law office with a heart.
Enter the hours and minutes of your consultation:
5  46
For 5 hours and 46 minutes, your bill is $3450.00
```

Call-by-Value Pitfall

- Common Mistake:
- Declaring parameter ‘again’ inside function:

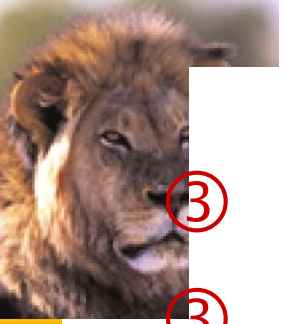
```
double fee(int hoursWorked, int minutesWorked)
{
        int quarterHours;  // local variable int
        minutesWorked    // NO!
}
```

- Compiler error results
- “Redefinition error…”

- Value arguments ARE like ‘local variables’
- But function gets them ‘automatically’

Call-By-Reference Parameters

- Used to provide access to caller’s actual argument
- Caller’s data can be modified by called function!
- Typically used for input function



- To retrieve data for caller
- Data is then ‘given’ to caller
- Specified by ampersand, &, after type in formal parameter list

Call-By-Reference Example

Display 4.2, page 138

Display 4.2 **Call-by-Reference Parameters** *(part 1 of 2)*

```
1  //Program to demonstrate call-by-reference parameters.
2  #include <iostream>
3  using namespace std;

4  void getNumbers(int& input1, int& input2);
5  //Reads two integers from the keyboard.

6  void swapValues(int& variable1, int& variable2);
7  //Interchanges the values of variable1 and variable2.

8  void showResults(int output1, int output2);
9  //Shows the values of variable1 and variable2, in that order.

10 int main( )
11 {
12     int firstNum, secondNum;

13     getNumbers(firstNum, secondNum);
14     swapValues(firstNum, secondNum);
15     showResults(firstNum, secondNum);
16     return 0;
17 }

18 void getNumbers(int& input1, int& input2)
19 {
20     cout << "Enter two integers: ";
21     cin >> input1
22         >> input2;
23 }

24 void swapValues(int& variable1, int& variable2)
25 {
26     int temp;

27     temp = variable1;
28     variable1 = variable2;
29     variable2 = temp;
30 }
31
32 void showResults(int output1, int output2)
33 {
34     cout << "In reverse order the numbers are: "
35          << output1 << " " << output2 << endl;
36 }
```

Call-By-Reference Example Cont'd

Display 4.2, page 138

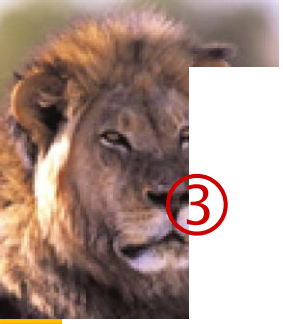
Display 4.2 **Call-by-Reference Parameters *(part 2 of 2)***

SAMPLE DIALOGUE

```
Enter two integers: 5 6
In reverse order the numbers are: 6 5
```

Call-By-Reference Details

③ What's really passed in?



- A ‘reference’ back to caller’s actual argument!
- Refers to memory location of actual argument
- Called ‘address’, which is a unique number referring to distinct place in memory

Constant Reference Parameters

- Reference arguments inherently ‘dangerous’
- Caller’s data can be changed
- Often this is desired, sometimes not

- To ‘protect’ data, & still pass by reference:
- Use const keyword
- void sendConstRef(const int &par1, const int &par2);
- Makes arguments ‘read-only’ by function
- No changes allowed inside function body

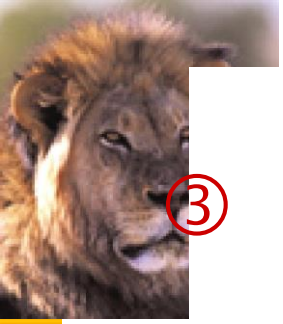
Parameters and Arguments

- Confusing terms, often used interchangeably ③ True meanings:
- Formal parameters

- In function declaration and function definition
- Arguments
- Used to ‘fill-in’ a formal parameter
- In function call (argument list)
- Call-by-value & Call-by-reference
- Simply the ‘mechanism’ used in plug-in process

Mixed Parameter Lists

- Can combine passing mechanisms



- Parameter lists can include pass-by-value and pass-by-reference parameters
- Order of arguments in list is critical:

  void mixedCall(int & par1, int par2, double & par3);
- Function call:

  mixedCall(arg1, arg2, arg3);
- arg1 must be integer type, is passed by reference
- arg2 must be integer type, is passed by value
- arg3 must be double type, is passed by reference

Choosing Formal Parameter Names

- Same rule as naming any identifier:
- Meaningful names!
- Functions as ‘self-contained modules’
- Designed separately from rest of program
- Assigned to teams of programmers
- All must ‘understand’ proper function use
- OK if formal parameter names are same as argument names

③ Choose function names with same rules